



UNIVERSIDADE ESTADUAL DE SANTA CRUZ
DEPARTAMENTO DE CIÊNCIAS EXATAS E TECNOLÓGICAS
CIÊNCIA DA COMPUTAÇÃO

ANA LUIZA OLIVEIRA DE FREITAS
YURI COUTINHO COSTA

BENCHMARK DE SUAVIZAÇÃO COM CUDA

Ilhéus - BA

2026

ANA LUIZA OLIVEIRA DE FREITAS

YURI COUTINHO COSTA

BENCHMARK DE SUAVIZAÇÃO COM CUDA

Relatório apresentado à Faculdade de Ciência da Computação da Universidade Estadual Santa Cruz como requisito parcial para a conclusão da disciplina de Processamento Paralelo, no semestre de 2026.1.

Orientador (a): Prof. Dr. Esbel Tomás Valero Orellana.

Ilhéus - BA

2026

1. INTRODUÇÃO

1.1 Objetivo

Otimizar um algoritmo de suavização/desfoque Gaussiano iterativo em imagens 2D, aplicando técnicas de paralelismo massivo por GPU com CUDA.

1.2 Conceitos Essenciais

1.2.1 Suavização/Desfoque Gaussiano

A suavização gaussiana é uma técnica de processamento de imagens que atua como um filtro passa-baixa, reduzindo ruídos e suavizando detalhes por meio de uma média ponderada baseada na distribuição normal. O processo utiliza um kernel quadrado de dimensão ímpar, cujos pesos são ditados pelo desvio padrão/sigma. Enquanto o tamanho do kernel define a área de vizinhança que influenciará o pixel central, o sigma controla a dispersão da "curva de sino" gaussiana; quanto maior o valor de sigma, maior será o peso dado aos pixels vizinhos e, conseqüentemente, mais intenso será o efeito de borrão.

Cada elemento do kernel segue uma função exponencial e, em seguida, é aplicada a normalização para que a soma total de seus pesos seja igual a 1, evitando alterações indesejadas no brilho da imagem. A aplicação prática ocorre via convolução, onde o kernel desliza sobre a imagem original para calcular o novo valor de cada pixel.

1.2.2 Paralelismo massivo em GPU

O paralelismo massivo em GPU é a capacidade da arquitetura da placa de vídeo executar milhares de operações simultaneamente. Diferente da CPU, que possui poucos núcleos, otimizados para processar tarefas sequenciais e complexas em alta velocidade, a GPU é estruturada com milhares de núcleos menores e mais simples. Quando um problema computacional pode ser dividido em partes totalmente independentes como o nosso do filtro de gauss, a GPU distribui o cálculo de forma simultânea entre seus milhares de núcleos. Dessa forma, em vez de resolver uma instrução após a outra de maneira linear, o hardware processa blocos inteiros de dados no mesmo ciclo de clock. Isso permite

resolver problemas de alta carga matemática e repetitiva em uma fração do tempo que um processador levaria.

1.2.3 Arquitetura CUDA

A arquitetura CUDA (Compute Unified Device Architecture) é uma plataforma de computação paralela desenvolvida pela NVIDIA que permite que a Unidade de Processamento Gráfico (GPU) realize cálculos complexos de propósito geral e não apenas para aplicações gráficas. Neste modelo de computação, são aplicados dois componentes principais: o *host*, que corresponde a CPU disponível no sistema, e o *device*, que faz referência a GPU. Assim, analogicamente, a memória do sistema associada a CPU é chamada de *memory host* e a memória associada a GPU, é a *memory device*.

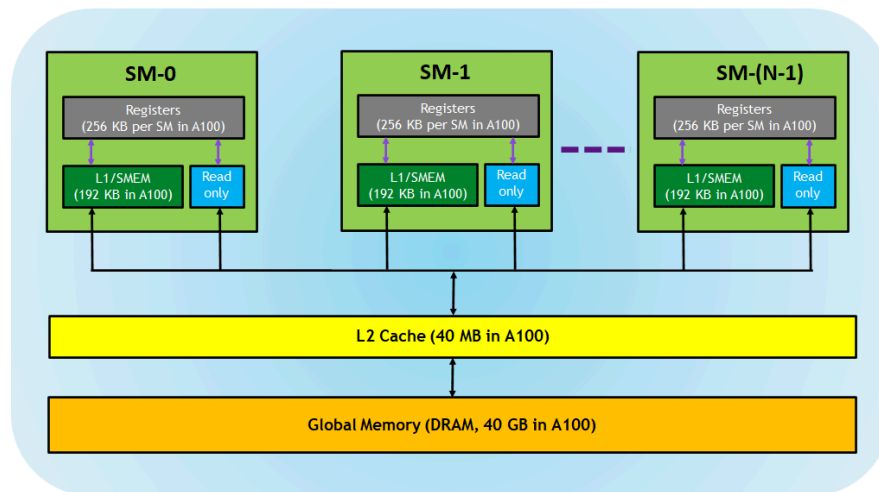
No fluxo de execução CUDA, é necessário que haja, inicialmente, a transferência de dados do *host* para o *device* para que ocorra o carregamento das funções na GPU e, em seguida, a execução. Com a conclusão dos cálculos, é preciso que os resultados sejam transferidos/copiados da memória do *device* para a memória do *host*.

Essas funções em C/C++ disparadas pelo *host* e executadas pelo *device* são chamadas de *kernels*. Diferente das funções sequenciais comuns, os *kernels* começam com o especificador de declaração `__global__` indicando que o *kernel* pode ser chamado tanto do *host* quanto do dispositivo, mas que pode ser executado k vezes em paralelo por k threads CUDA diferentes no *device*.

Além disso, para gerenciar o grande número de concorrências, a arquitetura organiza/agrupa as threads em blocos e organiza os blocos em grades. Assim, o kernel é executado como uma grade de blocos de threads.

1.2.4 Hierarquia de memória da GPU

Figura 1 - Hierarquia de memória em GPUs.



Fonte: NVIDIA (2020).

Para realizar a execução simultânea de milhares de threads sem atrapalhar o fluxo de dados, a arquitetura CUDA implementa uma hierarquia de memória, que é estruturada com base na velocidade e proximidade aos núcleos de processamento. Com a menor latência e maior velocidade de transmissão, encontram-se os Registradores, memórias privadas para cada thread que armazenam variáveis locais e índices de matrizes e são controladas pelo compilador.

Um nível abaixo dos registradores, temos a Memória Compartilhada, que é identificada pelo especificador de declaração `__shared__` e possui como escopo o bloco das threads, ou seja, todas as threads pertencentes a um mesmo bloco podem ler e escrever nessa memória.

Por fim, temos a Memória Global que representa o maior espaço de armazenamento disponível e também a maior latência da hierarquia, levando vários ciclos de clock para concluir uma única operação. Além disso, a memória global possui escopo total, podendo ser acessada por todas as threads de todos os blocos do Grid.

1.3 Importância da análise de desempenho em ambientes GPU

A análise de desempenho é a etapa que valida a eficácia da paralelização, transformando dados de tempo em métricas quantificáveis. Em processamento paralelo, essa análise permite determinar o *Speedup*, que mede o ganho de

velocidade obtido ao distribuir a carga de trabalho entre os múltiplos processos em relação à execução puramente sequencial, e a *Eficiência* do sistema, indicando o quão bem os recursos do hardware, como os núcleos de performance e processador, estão sendo aproveitados para resolver o problema proposto. Em ambientes de computação acelerada por GPU, a análise de desempenho se torna ainda mais necessária devido à natureza paralela e a velocidade da GPU, que podem acabar mascarando implementações ineficientes. Além disso, essa análise pode ser uma forma de encontrar o ponto ótimo de operação do hardware, equilibrando o custo de transferência de dados com o poder computacional disponível. Por fim, é importante salientar a necessidade da análise de desempenho para mapear a escalabilidade do sistema, permitindo avaliar se a arquitetura respeita os limites de ganho de velocidade impostos pela Lei de Amdahl.

2. METODOLOGIA

2.1 Descrição da implementação paralela com CUDA

2.1.1. Kernels CUDA

O kernel CUDA desenvolvido foi o `__global__ void apply_convolution_cuda()`, que adota o mapeamento de uma thread por pixel, o que significa que cada thread vai processar exclusivamente um ponto (x,y) da imagem. Cada thread determina sua coordenada global de linha e coluna na matriz de pixels por meio das variáveis do CUDA, combinando o índice do bloco atual dentro da grade (blockIdx) com as dimensões dos blocos (blockDim) e a posição exata da thread dentro do seu bloco (threadIdx). Tendo em vista que, nesse caso, tratam-se de matrizes 2D, cada variável CUDA possui duas subcoordenadas .x e .y que são utilizadas para encontrar a coluna e a linha de cada thread na imagem, respectivamente.

Além disso, como as dimensões da imagem podem não ser múltiplos exatos do tamanho fixado para os blocos de threads, foi implementada uma estrutura condicional para garantir que apenas as threads cujos índices globais estejam dentro dos limites de altura e largura possam realizar operações.

2.1.2. Estratégia de mapeamento de threads

A configuração das dimensões dos blocos de threads foi implementada de forma dinâmica, permitindo sua variação via passagem de parâmetros (como 8x8, 16x16 e 32x32) para fins de análise de desempenho. Tomando como referência a topologia intermediária de 16x16 (256 threads por bloco), essa escolha justifica-se pelo alinhamento exato com o tamanho do Warp, unidade fundamental de execução de 32 threads da GPU. Além disso, blocos de tamanho médio evitam o esgotamento precoce dos registradores internos dos multiprocessadores. Isso permite que o agendador de hardware aloque múltiplos blocos simultaneamente no mesmo chip e execute um revezamento eficiente para ocultar a latência das buscas na memória global.

A partir da dimensão definida via parâmetro, o tamanho da grade de execução é calculado dinamicamente pela divisão da largura e da altura da imagem pelas respectivas dimensões do bloco escolhido. Essa abordagem garante que a grade mude de escala proporcionalmente e cubra todo o domínio do problema sem deixar pixels de fora. Utilizando o bloco base de 16x16 como exemplo, a malha gerada varia de 1.024 blocos para imagens de 512x512 até 65.536 blocos para resoluções de 4096x4096, conforme demonstrado na Tabela 1.

Tabela 1 - Configuração de blocos e grades por resolução de imagem

Resolução	Dimensão dos Blocos	Altura	Largura	Total de blocos
512px	16 x 16	$512/16 = 32$	$512/16 = 32$	$32 \times 32 = 1024$
1024px	16 x 16	$1024/16 = 64$	$1024/16 = 64$	$64 \times 64 = 4096$
2048px	16 x 16	$2048/16 = 128$	$2048/16 = 128$	$128 \times 128 = 16.384$
4096px	16 x 16	$4096/16 = 256$	$4096/16 = 256$	$256 \times 256 = 65.536$

Fonte: Autores (2026)

2.1.3. Uso de memória da GPU

A Memória Global foi utilizada para armazenar a matriz original e o buffer de saída por oferecer grande capacidade de armazenamento na VRAM da GPU, apesar de apresentar alta latência devida a sua distância com relação ao chip principal. Para compensar o custo das operações da convolução, a máscara foi alocada na Memória Constante (`__constant__ double d_kernel[MAX_MASK_SIZE]`), um espaço de leitura de alta velocidade que possui uma cache dedicada dentro de cada Multiprocessador de Fluxo (SM), permitindo que todas as threads de um *Warp* leiam os mesmos coeficientes do filtro simultaneamente em um único ciclo de clock via *broadcast*.

2.1.4. Transferências de dados host ↔ device

Inicialmente, é realizada a transferência dos dados da imagem do *host* para o *device* através do barramento PCI Express. A função utilizada é a `cudaMemcpy()` com o parâmetro `cudaMemcpyHostToDevice`. Com a transferência concluída, ocorre o processamento de dados na GPU e realização de cálculos. Após a execução de todas as iterações, acontece uma nova transferência de dados via barramento para resgatar a matriz final de volta para o *host* com a função `cudaMemcpy()` com o parâmetro `cudaMemcpyDeviceToHost`.

2.1.5. Funções CUDA utilizadas

As funções CUDA utilizadas foram:

- `CudaMalloc()`: realiza alocação dinâmica de memória do *device*. Como parâmetros, exige um ponteiro duplo para armazenar o endereço de memória e o tamanho em bytes;
- `CudaMemcpy()`: realiza a transferência de dados entre *host* e *device*. A direção do movimento dos dados é definida pelo parâmetro de controle escolhido, sendo `cudaMemcpyHostToDevice` para transferir a imagem original para a GPU e o `cudaMemcpyDeviceToHost` para transferir os resultados calculados de volta à CPU;

- *CudaMemcpyToSymbol()*: realiza a transferência de dados da memória do *host* para variáveis globais ou constantes alocadas na memória do *device*. Na implementação, a função foi utilizada para transferir a máscara para a memória constante da GPU.
- *CudaFree()*: realiza a desalocação e a liberação dos blocos de memória alocados com a função *CudaMalloc()*.

2.1.6. Tratamento das bordas da imagem

O tratamento de bordas foi realizado dentro das instruções do kernel de forma implícita através da técnica de *padding clamp* ou preenchimento por repetição, que ao invés de inventar um preenchimento no espaço fora da imagem, estica a última linha ou coluna existente.

Dessa forma, na implementação, quando a thread tenta ler pixels vizinhos para aplicar a máscara, os índices calculados de linha e coluna são checados pela estrutura condicional. Caso o kernel do filtro se desloque para fora dos limites da imagem, ou seja, tenha um índice negativo ou superior à altura e largura máximas, o índice assume o valor limite da borda interna correspondente.

2.2 Descrição do hardware utilizado nos testes

2.2.1. Especificação completa do hardware da GPU

- Modelo: NVIDIA GeForce RTX 4070 Laptop GPU (Codinome do chip: AD106)
- Arquitetura: Ada Lovelace (Compute Capability 8.9)
- Quantidade de SMs (Streaming Multiprocessors): 36 SMs, totalizando 4.608 núcleos CUDA, 144 Tensor Cores e 36 RT Cores.
- Memória Global Disponível: 8 GB GDDR6 (interface de 128-bit)

2.2.2. Especificação do Host (CPU)

Modelo: I7 14700HX

- Número de Núcleos: 20

- Memória RAM: 16

2.2.2. Ambiente de Software e Compilação

- Sistema Operacional: Windows 11
- Versão do Driver CUDA: 13.3
- Versão do Compilador (nvcc): 13.3 (Build 13.3.73)

2.4 Procedimentos para medição de tempo

Para a medição do tempo foi utilizado o CUDA Events, um recurso nativo que faz parte da API de tempo de execução do CUDA, que possui um conjunto de funções e estruturas disponibilizadas pela biblioteca da NVIDIA para interagir com a GPU. Na prática, ele permite medir o tempo decorrido entre pontos específicos da execução do programa.

O *cudaEvent_t* é o tipo de dado que guarda o identificador do evento, mas são as funções CUDA Events que permitem a medição real do tempo. As funções utilizadas foram:

- *cudaEventCreate()* para criar o evento;
- *cudaEventRecord()* para marcar o evento na linha de execução;
- *cudaEventSynchronize()* que espera o final do evento;
- *cudaEventElapsedTime()* que mede o tempo exato decorrido entre o registro de dois eventos na GPU;
- *cudaEventDestroy()* que deleta o evento da memória.

O tempo medido inclui apenas a execução do kernel, desprezando as transferências de dados entre host e device para que seja possível avaliar a qualidade do código paralelo e da arquitetura da GPU com maior precisão, demonstrando o ganho real de velocidade sem a interferência da latência de comunicação do barramento.

2.5 Parâmetros utilizados nos testes de desempenho

Tabela 2 - Configurações dos testes realizados

Variável	Valores Testados
Resoluções das Imagens	512px, 1024px, 2048px, 4096px

Kernel	3x3, 5x5
Número de iterações	100,500,1000
Tamanho do Bloco	8,16,32

Fonte: Autores (2026)

Cada configuração foi executada 3 vezes, e para o resultado final do teste, foi feita a média entre as execuções realizadas.

Total de execuções: $4 \times 2 \times 3 \times 3 \times 3 = 216$ vezes.

2.6 Métricas utilizadas para avaliação

- Tempo de Execução: O tempo de execução do kernel.
- Speedup: A razão entre o tempo de execução sequencial e o tempo de execução paralelo. Indica o ganho de velocidade;
- Eficiência: Para programação paralela com CUDA, a eficiência é discutida em termos de ocupação da GPU, que mede a capacidade do código de manter os processadores físicos da GPU ocupados, evitando que os núcleos fiquem ociosos. Dessa forma, um valor de eficiência próximo a 0 indica que o sistema sofreu com gargalos, como a latência no acesso à memória global ou threads ociosas nas barreiras de sincronização;
- Discussão sobre Escalabilidade: Análise de como o desempenho se comporta ao aumentar o tamanho do problema;
- Análise comparativa com as soluções das etapas anteriores (OpenMP e MPI), quando aplicável.

3. RESULTADOS

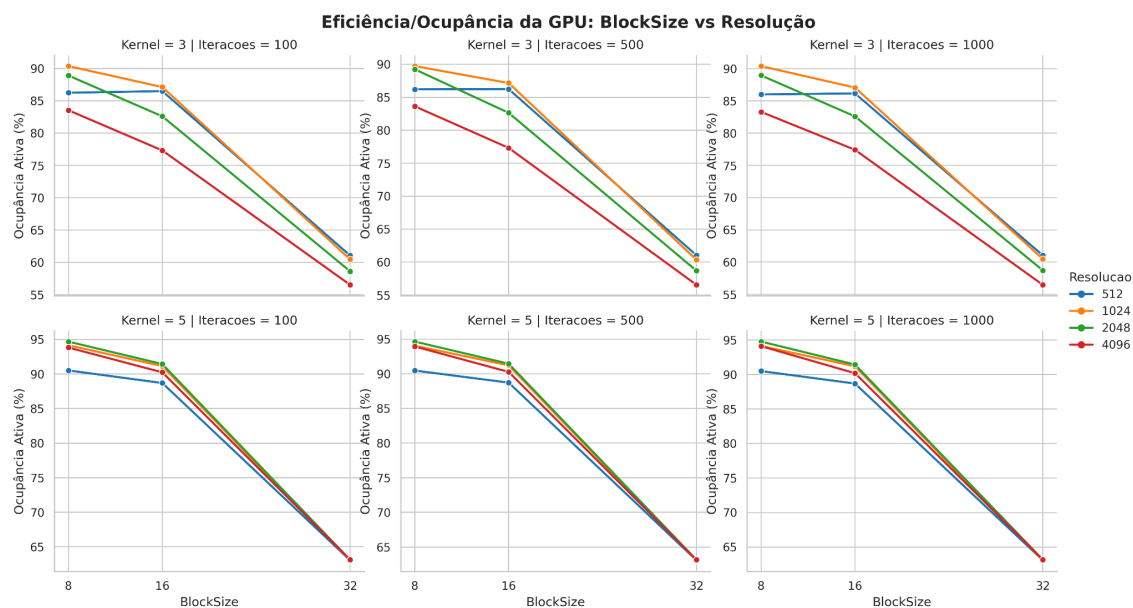
3.1 Teste de Corretude

Primeiro utilizamos o código em python com a matriz simples e os parâmetros de exemplo e comparamos com o resultado gerado pelo código serial em C, assim foi possível verificar que os resultados eram idênticos. Em seguida, elaboramos um *shell script* para executar o código serial e paralelo com diferentes resoluções, tamanho de kernel, quantidade de iterações e tamanho do bloco. Ao mesmo tempo, comparamos a saída da imagem gerada pelo paralelo

com a gerada pelo código serial, utilizando os mesmos parâmetros. Essa comparação foi feita utilizando o *compare* do *imagemagick* tolerando um erro de 1%. Dessa forma, foi possível verificar que não há diferença nos resultados produzidos, já que, em ambos, foi constatado que as imagens eram idênticas.

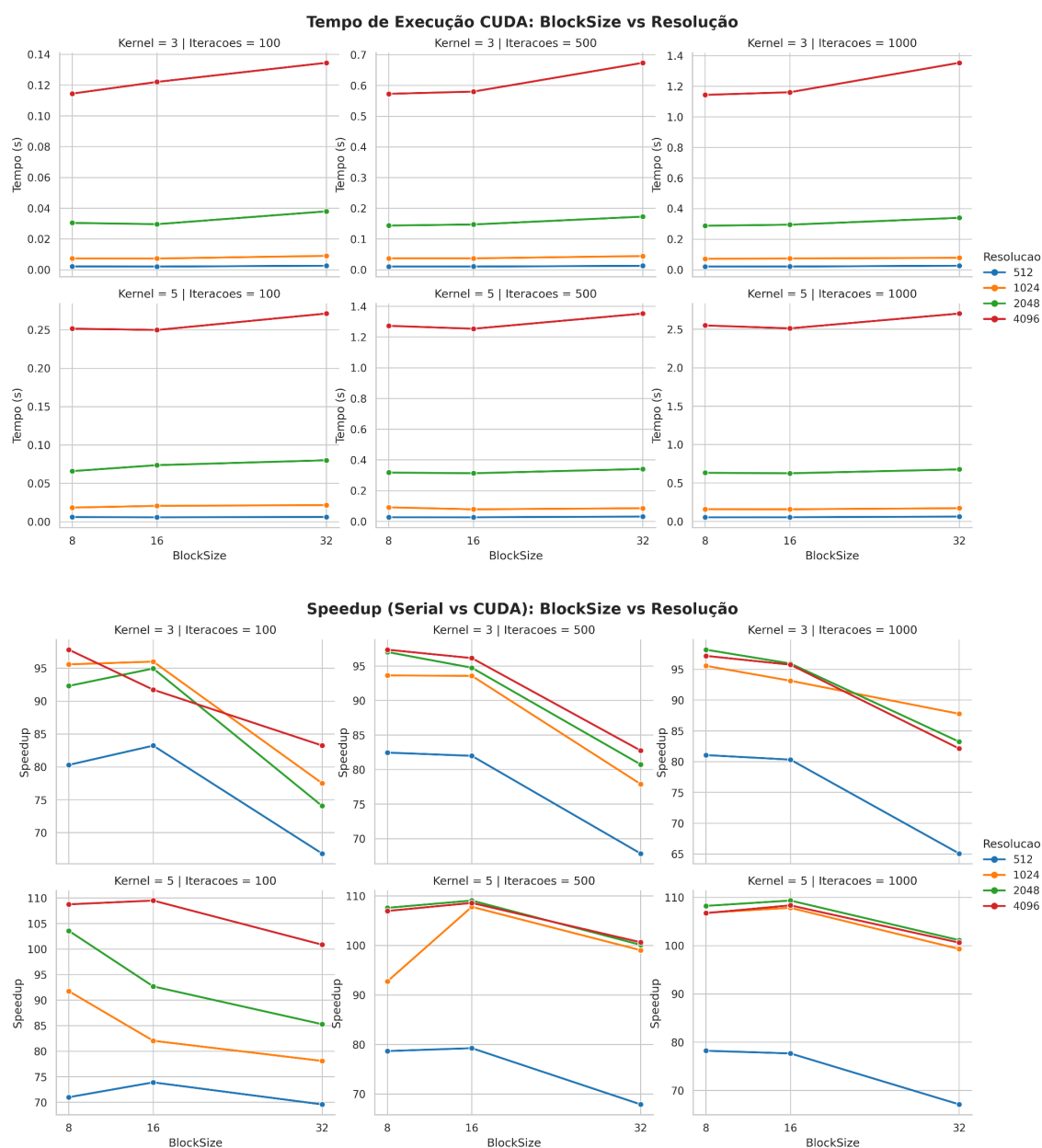
3.2 Gráficos e/ou Tabelas

Figura 2 - Gráfico de Ocupância da GPU: BlockSize vs Resolução



Fonte: Autores (2026)

Figura 3 - Gráfico de Tempo de Execução CUDA: BlockSize vs Resolução



3.3 Cálculos de Speedup e Eficiência

Resolução	Kernel	Iterações	Block Size	TSeq (s)	TCUDA (s)	Ocupação	Speedup
512 × 512	3x3	100	8 × 8	0,1734	0,0022	86,25%	80,31x
	3x3	100	16 × 16	0,1734	0,0021	86,50%	83,24x
	3x3	100	32 × 32	0,1734	0,0026	61,06%	66,83x
	3x3	500	8 × 8	0,8653	0,0105	86,21%	82,47x

	3x3	500	16 × 16	0,8653	0,0106	86,25%	82,00x
	3x3	500	32 × 32	0,8653	0,0128	61,02%	67,85x
	3x3	1000	8 × 8	1,7267	0,0213	86,01%	81,07x
	3x3	1000	16 × 16	1,7267	0,0215	86,17%	80,34x
	3x3	1000	32 × 32	1,7267	0,0265	61,04%	65,08x
	5x5	100	8 × 8	0,4287	0,006	90,52%	71,01x
	5x5	100	16 × 16	0,4287	0,0058	88,70%	73,91x
	5x5	100	32 × 32	0,4287	0,0062	63,05%	69,60x
	5x5	500	8 × 8	2,1381	0,0272	90,47%	78,67x
	5x5	500	16 × 16	2,1381	0,027	88,72%	79,27x
	5x5	500	32 × 32	2,1381	0,0315	63,08%	67,91x
	5x5	1000	8 × 8	4,194	0,0536	90,50%	78,23x
	5x5	1000	16 × 16	4,194	0,054	88,69%	77,68x
	5x5	1000	32 × 32	4,194	0,0625	63,06%	67,11x
1024 × 1024	3x3	100	8 × 8	0,7001	0,0073	90,37%	95,59x
	3x3	100	16 × 16	0,7001	0,0073	87,12%	96,01x
	3x3	100	32 × 32	0,7001	0,009	60,47%	77,52x
	3x3	500	8 × 8	3,4783	0,0371	89,73%	93,64x
	3x3	500	16 × 16	3,4783	0,0372	87,17%	93,58x
	3x3	500	32 × 32	3,4783	0,0446	60,35%	77,90x
	3x3	1000	8 × 8	6,8918	0,0721	90,41%	95,57x
	3x3	1000	16 × 16	6,8918	0,074	87,07%	93,15x
	3x3	1000	32 × 32	6,8918	0,0785	60,47%	87,78x
	5x5	100	8 × 8	1,6967	0,0185	94,17%	91,77x
	5x5	100	16 × 16	1,6967	0,0207	91,14%	82,06x
	5x5	100	32 × 32	1,6967	0,0217	63,06%	78,11x
	5x5	500	8 × 8	8,5054	0,0917	94,06%	92,75x
	5x5	500	16 × 16	8,5054	0,0789	91,21%	107,81x
	5x5	500	32 × 32	8,5054	0,0859	63,08%	99,05x

	5x5	1000	8 × 8	17,0446	0,1596	94,14%	106,82x
	5x5	1000	16 × 16	17,0446	0,1581	91,19%	107,84x
	5x5	1000	32 × 32	17,0446	0,1716	63,05%	99,31x
2048 × 2048	3x3	100	8 × 8	2,8123	0,0305	88,91%	92,31x
	3x3	100	16 × 16	2,8123	0,0296	82,60%	94,96x
	3x3	100	32 × 32	2,8123	0,038	58,62%	74,07x
	3x3	500	8 × 8	13,9746	0,144	89,25%	97,03x
	3x3	500	16 × 16	13,9746	0,1475	82,64%	94,76x
	3x3	500	32 × 32	13,9746	0,1731	58,68%	80,75x
	3x3	1000	8 × 8	28,2929	0,2881	88,97%	98,19x
	3x3	1000	16 × 16	28,2929	0,2949	82,60%	95,95x
	3x3	1000	32 × 32	28,2929	0,3399	58,71%	83,24x
	5x5	100	8 × 8	6,8328	0,066	94,69%	103,59x
	5x5	100	16 × 16	6,8328	0,0737	91,44%	92,70x
	5x5	100	32 × 32	6,8328	0,0801	63,12%	85,30x
	5x5	500	8 × 8	34,2022	0,3178	94,64%	107,61x
	5x5	500	16 × 16	34,2022	0,3136	91,45%	109,08x
	5x5	500	32 × 32	34,2022	0,3416	63,15%	100,13x
	5x5	1000	8 × 8	68,4773	0,6326	94,78%	108,25x
	5x5	1000	16 × 16	68,4773	0,6262	91,45%	109,35x
	5x5	1000	32 × 32	68,4773	0,6771	63,15%	101,14x
4096 × 4096	3x3	100	8 × 8	11,2007	0,1145	83,54%	97,82x
	3x3	100	16 × 16	11,2007	0,1221	77,32%	91,74x
	3x3	100	32 × 32	11,2007	0,1345	56,52%	83,26x
	3x3	500	8 × 8	55,7836	0,5729	83,63%	97,37x
	3x3	500	16 × 16	55,7836	0,5802	77,32%	96,15x
	3x3	500	32 × 32	55,7836	0,674	56,54%	82,76x
	3x3	1000	8 × 8	111,163	1,1438	83,30%	97,19x

	3x3	1000	16 × 16	111,163	1,1609	77,42%	95,75x
	3x3	1000	32 × 32	111,163	1,3536	56,46%	82,13x
	5x5	100	8 × 8	27,3638	0,2516	93,83%	108,78x
	5x5	100	16 × 16	27,3638	0,2499	90,24%	109,52x
	5x5	100	32 × 32	27,3638	0,2713	63,15%	100,87x
	5x5	500	8 × 8	136,17	1,2729	93,94%	106,98x
	5x5	500	16 × 16	136,17	1,2539	90,27%	108,60x
	5x5	500	32 × 32	136,17	1,3529	63,16%	100,65x
	5x5	1000	8 × 8	272,245	2,5504	94,12%	106,75x
	5x5	1000	16 × 16	272,245	2,5127	90,19%	108,35x
	5x5	1000	32 × 32	272,245	2,7055	63,16%	100,63x

3.4 Análise de Ocupância e Configuração de Blocos

Para compreender a eficiência da implementação em CUDA, conduziu-se uma análise de Ocupância utilizando o NVIDIA Nsight Compute. A ocupância reflete o percentual de capacidade de processamento efetivamente ativa na GPU. Foram testadas três configurações de dimensão de blocos: 8x8, 16x16 e 32x32. O comportamento do hardware demonstrou variações críticas à medida que as dimensões do bloco aumentaram. De forma geral, aumentar o tamanho do bloco resultou em tempos de execução maiores, fazendo do 8x8 a configuração mais rápida e com maior ocupância para o kernel 3x3. Isso fica evidente ao utilizar o tamanho máximo de bloco (32x32, com 1024 threads). A quantidade de threads simultâneas por bloco é tão grande que exige muitos recursos do núcleo de processamento. O resultado é que o gerenciamento delas se torna um gargalo: se essas threads precisarem aguardar os dados da memória, a GPU não tem espaço livre para encaixar outros blocos, fazendo com que o hardware fique ocioso. Essa alta ociosidade derrubou a ocupância para cerca de 60% e causou o pior tempo de execução em todos os testes. A única exceção a essa regra ocorreu ao utilizar o kernel 5x5, onde o bloco 16x16 superou o 8x8. Isso aconteceu porque a carga de cálculo do kernel 5x5 é muito maior. Em blocos pequenos (8x8), a placa perde eficiência gerenciando o excesso de

bloquinhos e processando dados repetidos nas bordas de cada um. Para esse cenário mais pesado, o bloco 16x16 atuou como um ponto ótimo: ele minimizou o esforço do escalonador e manteve threads suficientes ativas para evitar a ociosidade do hardware.

4. DISCUSSÃO

4.1 Análise da corretude das implementações

Como nosso teste de corretude não gerou nenhum erro e/ou imagem diferente, podemos concluir que a implementação paralela está correta. A validação visual (via *compare*) confirmam que o particionamento do domínio não introduziu condições de corrida ou erros de sincronização na escrita da imagem de saída.

4.2 Comparação entre a versão sequencial e a paralela com CUDA

Com base nos dados gerados é possível analisar que a versão paralela com CUDA resultou em uma redução drástica do tempo e ao aumentar o kernel o speed up e o tempo aumentaram, o que significa que houve um ganho significativo, pois o tempo do serial aumentou muito mais em relação ao tempo cuda ao aumentar o kernel. Quando comparamos os speed up de cada resolução, percebemos que a resolução de 512x512x tem um speed up muito distante das demais. Essa distância ocorre pois essa resolução é tão pequena que o custo para gerenciar, preparar e enfileirar as 1000 chamadas na CPU torna-se proporcionalmente grande em relação ao tempo que a GPU gasta calculando, já para resoluções maiores a GPU passa mais tempo calculando de verdade e o tempo de preparo se torna insignificante.

4.3 Comparação com as Etapas 1 e 2

As duas primeiras etapas implementadas avaliando o paralelismo em CPU, OpenMp e MPI, foram executadas no Apple M5. Os teste CUDA, por outro lado, foram realizados no Acer Predator Helios Neo 16 com uma GPU NVIDIA. Tendo em vista a disparidade nos ambientes de execução, não é possível fazer uma análise comparativa direta dos resultados, principalmente no que diz respeito ao tempo de execução. Entretanto, a análise das tendências de

escalabilidade, comportamento dos algoritmos e eficiência computacional da arquitetura permanece válida.

Enquanto as abordagens baseadas em CPU apresentaram uma escalabilidade limitada ao número de núcleos disponíveis no processador, sofrendo perdas de desempenho associadas ao overhead de sincronização de memória e ou de comunicação entre processos, a GPU apresentou uma alta capacidade de escalabilidade.

Além disso, ao analisar e comparar o SpeedUp das etapas implementadas, é possível perceber a discrepância entre paradigmas de computação baseados em CPU e GPU. Mesmo considerando que os testes foram realizados em hardwares distintos, o comportamento proporcional do SpeedUp evidencia como os modelos anteriores não se aproximaram da eficiência da GPU. Enquanto as etapas anteriores apresentaram ganhos de velocidades contidos e similares, a etapa em CUDA alcançou uma execução 109,52x mais rápida que a execução sequencial, o que demonstra a sua superioridade computacional.

4.4 Discussão de gargalos e limitações encontradas

4.4.1. Memory-bound vs Compute-bound

O algoritmo de suavização gaussiana, fundamentado na operação de convolução 2D, classifica-se primariamente como um problema Memory-bound. Isso indica que o desempenho geral do sistema é restringido pela velocidade com que os dados trafegam da memória de vídeo para os núcleos de processamento. A explicação para esse fator é a baixa intensidade aritmética da operação. Ao calcular o valor de um único pixel com uma máscara 5x5, cada thread precisa realizar 25 leituras na memória da imagem e aproximadamente o mesmo número de operações aritméticas. Como a placa de vídeo utilizada tem uma alta capacidade de processamento, essas instruções são executadas em poucos ciclos de clock. Portanto, os núcleos tendem a ficar ociosos aguardando a chegada dos dados provenientes da Memória Global, que possui uma latência significativamente maior.

4.4.2. Overhead de transferência host ↔ device

Como o foco deste trabalho foi avaliar estritamente a capacidade de processamento bruto da GPU em comparação ao código serial, o tempo gasto com a transferência de dados via barramento PCIe não foi incluído nas métricas. Na instrumentação do código, os marcadores de tempo (`cudaEvent_t`) foram posicionados intencionalmente para isolar o laço iterativo, iniciando *após* o envio da imagem inicial (`cudaMemcpy Host para Device`) e finalizando *antes* da cópia do resultado de volta para a memória principal (`Device para Host`). Consequentemente, o *overhead* de comunicação entre a memória RAM e a VRAM foi descartado. Essa decisão garante que os expressivos ganhos de Speedup relatados reflitam puramente o poder de aceleração matemática dos núcleos CUDA, sem a interferência do gargalo de comunicação da placa-mãe.

4.5 Impacto da configuração de blocos e grades no desempenho

Ao analisar a divisão de blocos, nota-se que não existe uma configuração única ideal para todos os casos, pois o desempenho varia diretamente com o tamanho do kernel utilizado. Em todos os cenários, a configuração máxima permitida (32x32) apresentou os piores resultados. Agrupar tantas threads em um único bloco engessa a distribuição do trabalho e esgota rapidamente os recursos físicos dos núcleos, fato que foi analisado em profundidade na Seção 3.4 (Análise de Ocupância e Configuração de Blocos). A eficiência real é definida no comparativo entre os blocos 8x8 e 16x16. Quando utilizamos o kernel menor (3x3), o bloco 8x8 apresenta o melhor desempenho. Como a máscara 3x3 exige poucos cálculos por pixel, a placa de vídeo consegue lidar muito bem com a alta quantidade de microblocos, aproveitando ao máximo a ocupação do hardware sem gargalos no gerenciamento. Por outro lado, quando a carga matemática aumenta com o kernel 5x5, a situação se inverte e o bloco 16x16 passa a ser o vencedor. O kernel 5x5 exige muito mais processamento e memória. Nesse cenário mais pesado, gerenciar milhares de microblocos 8x8 sobrecarrega o controle da placa. O particionamento em 16x16 resolve esse problema, pois balanceia a grade de forma ideal, mantendo o sistema ocupado sem estourar o limite de gerenciamento.

4.6 Sugestões de melhorias ou otimizações futuras

Para implementações futuras, poderia ser utilizada a Memória Compartilhada para substituir ou complementar o uso da Memória Global e constante. Além disso, visando uma comparação mais direta e precisa, a principal meta futura seria a padronização integral do ambiente de testes, eliminando possíveis variáveis externas e contribuindo para resultados mais completos e confiáveis para estudo e pesquisa. Por fim, tendo em vista os grandes esforços e avanços em cada uma das etapas do projeto desenvolvido, seria interessante o desenvolvimento de uma implementação híbrida combinando as tecnologias estudadas para elevar a eficiência e a escalabilidade do projeto.

5. CONCLUSÃO

5.1 Principais aprendizados do projeto

Através do desenvolvimento do projeto, foi possível compreender a estrutura e funcionamento da GPU, a arquitetura CUDA e os seus mecanismos de otimização que foram essenciais para a melhora do desempenho da GPU. Durante o desenvolvimento do código, um dos principais aprendizados foram as funções da API do CUDA que foram responsáveis pelo bom funcionamento do código. Além disso, o entendimento dos novos conceitos de eficiência e ocupação foram essenciais para a análise de aspectos muitas vezes subestimados como a configuração dos tamanhos dos blocos e o uso de memória da GPU.

5.2 Considerações sobre desempenho obtido

Os resultados obtidos evidenciam que um maior desempenho e SpeedUp estão diretamente relacionados com o volume do problema. Em resoluções menores, como 512x512, o SpeedUp se manteve na faixa de 70x a 83x, o que ainda não representava o melhor desempenho considerando o seu poder computacional. Entretanto, foi em resoluções massivas, como 4096x4096, que a GPU obteve o melhor resultado de todos os testes já realizados, sendo 109,52x mais rápida que a versão sequencial. Com isso, é possível perceber a

necessidade de utilizar uma quantidade massiva de dados durante a execução para compensar os custos de inicialização e otimizar os resultados.

Outra consideração fundamental está relacionada às limitações físicas impostas pelo tamanho dos blocos de threads. Os dados mostram que maiores taxas de ocupação, blocos de 8x8 e 16x16, levaram a menores tempos de processamento, o que não acontece com tamanhos maiores, que acabam esgotando os registradores. Dessa forma, é notável o entendimento de que o desempenho máximo em CUDA depende do equilíbrio entre a quantidade de dados por bloco, a quantidade de dados e a preservação dos recursos físicos da GPU.

5.3 Considerações sobre os desafios encontrados

Inicialmente, o principal desafio foi encontrar uma maneira de diminuir a latência de transferência de dados decorrente da distância física entre a memória principal e a VRAM através do barramento. Nas execuções com maiores iterações, o acesso repetido à memória global causava grandes reduções no desempenho. Para mitigar esses efeitos, foi utilizada a Memória Constante que garantia leituras rápidas através do *broadcast*.

Além disso, considerando a necessidade de utilizar um hardware com uma GPU da NVIDIA, foi preciso mudar o ambiente de execução de testes para um hardware cedido por um integrante de outra equipe. Essa dependência impôs restrições ao cronograma de testes, limitando à execução a horários específicos.

6. REFERÊNCIAS

ORELLANA, Esbel T. Valero. *Aula 12 - Programação para GPGPUs: CUDA*. Ilhéus: UESC, 2026. Slides de aula da disciplina CET107 - Processamento Paralelo.

ORELLANA, Esbel T. Valero. *Aula 13 - Programação para GPGPUs: CUDA*. Ilhéus: UESC, 2026. Slides de aula da disciplina CET107 - Processamento Paralelo.

NVIDIA Corporation. CUDA C++ Programming Guide. Disponível em: <<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>>. Acesso em: 02 jul. 2026.

NVIDIA. CUDA Refresher: The CUDA Programming Model. In: NVIDIA Developer Blog. [S. l.], 23 jun. 2020. Disponível em:

<https://developer.nvidia.com/blog/cuda-refresher-cuda-programming-model/>. Acesso em: 2 jul. 2026

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Nós, Ana Luiza Oliveira de Freitas e Yuri Coutinho Costa, regularmente matriculados no curso de Ciência da Computação, declaramos para os devidos fins que, na elaboração do projeto intitulado Benchmark de Suavização com CUDA, submetido como requisito para a disciplina DEC107 — Processamento Paralelo, adotamos as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

() Não utilizamos ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(x) Utilizamos ferramentas de Inteligência Artificial como recursos de apoio.

As ferramentas utilizadas foram: Google Gemini. O uso se deu exclusivamente nas seguintes etapas do projeto:

1. Estruturação inicial de ideias e revisão bibliográfica;
2. Otimização da função de leitura do arquivo;
3. Auxílio na depuração (debugging) de blocos específicos de código;

Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmando que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus - BA, 04 de Julho de 2026

[Ana Luiza Oliveira de Freitas](#)- 202310767

[Yuri Coutinho Costa](#)- 202311127